

Relazione di Calcolo Numerico

Esercizio n.1

Christian Barbarossa

2 Luglio 2026

1 Introduzione Teorica

L'obiettivo del presente esercizio è lo studio dell'approssimazione di una funzione continua $f(x) = e^{-x} \cos(2x)$ nell'intervallo $[a, b] = [-2, 2]$ mediante un polinomio interpolante d'ordine $n = 8$, analizzando la scelta dei nodi di campionamento.

Considerando un set di $n + 1$ nodi distinti $x_0, x_1, \dots, x_n$, l'interpolazione polinomiale nella base monomiale consiste nel determinare l'unico polinomio:

$$P_n(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n \quad (1)$$

tale che $P_n(x_i) = f(x_i) = y_i$ per ogni $i = 0, \dots, n$. Tale condizione si traduce nella risoluzione di un sistema lineare di equazioni la cui matrice dei coefficienti prende il nome di **Matrice di Vandermonde** (V):

$$Va = y \implies \begin{pmatrix} 1 & x_0 & x_0^2 & \dots & x_0^n \\ 1 & x_1 & x_1^2 & \dots & x_1^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^n \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{pmatrix} \quad (2)$$

1.1 Instabilità e Pivot Totale

Le matrici di Vandermonde basate su nodi equispaziati sono notoriamente *mal condizionate*: l'indice di condizionamento $K(V)$ cresce esponenzialmente al crescere del grado n . Al fine di mitigare la propagazione degli errori di arrotondamento intrinseci all'aritmetica di macchina durante la fase di eliminazione gaussiana, viene implementata manualmente la **Fattorizzazione LU con Pivot Totale**.

A ogni passo k dell'algoritmo, la ricerca del pivot viene estesa a tutta la sottomatrice rimanente $A(k : n, k : n)$. Questo processo richiede lo scambio sia delle righe (equazioni) sia delle colonne (incognite), definendo due matrici di permutazione P e Q tali che:

$$PVQ = LU \quad (3)$$

La successiva risoluzione del sistema avviene in quattro fasi sequenziali:

$$1. \quad y_{\text{perm}} = Py \quad (4)$$

$$2. \quad Lz = y_{\text{perm}} \quad (\text{Sostituzione in avanti}) \quad (5)$$

$$3. \quad U\tilde{a} = z \quad (\text{Sostituzione all'indietro}) \quad (6)$$

$$4. \quad a = Q\tilde{a} \quad (\text{Ripristino dell'ordine originario delle incognite}) \quad (7)$$

1.2 Il Fenomeno di Runge e i Nodi di Chebyshev

L'utilizzo di nodi equispaziati per l'interpolazione globale di grado elevato introduce forti oscillazioni distruttive in prossimità degli estremi dell'intervallo, fenomeno noto come **Fenomeno di Runge**. Per ovviare a tale instabilità matematica, si ricorre ai **Nodi di Chebyshev**, i quali addensano i punti di campionamento verso i bordi dell'intervallo secondo la legge:

$$x_k = \frac{a+b}{2} + \frac{b-a}{2} \cos\left(\frac{2k+1}{2n+2}\pi\right), \quad k = 0, \dots, n \quad (8)$$

Questo minimizza l'errore massimo di interpolazione.

2 Codice Sorgente Python

Di seguito viene riportato lo script Python completo sviluppato per l'esecuzione dei calcoli, la generazione dei grafici e l'estrazione degli indici di stabilità numerica.

```
1 import numpy as np
2 import scipy.linalg as las
3 import matplotlib.pyplot as plt
4 from matplotlib.widgets import CheckButtons # libreria per implementare i bottoni
5 from tabulate import tabulate # utilizzato per la formattazione della tabella finale
6
7 # =====
8 # Punto 1: Definizione della funzione e dei punti equispaziati
9 # =====
10
11 def f(x):
12     # La funzione f(x) = e-x * cos(2x) da interpolare
13     return np.exp(-x) * np.cos(2*x)
14
15 # vado a creare l'array di 9 punti equispaziati nell'intervallo [-2, 2]
16 n_punti = 9
17 x_eq = np.linspace(-2, 2, n_punti) # mi restituisce l'array di punti
18 print("Nodi equidistanti: " + str(x_eq)) # stampo i punti sul terminale
19 y_eq = f(x_eq)
20
21 # =====
22 # Punto 2 & 3: Base monomiale, Matrice di Vandermonde e Fattorizzazione LU
23 # =====
24
25 # Costruisco la Matrice di Vandermonde per i punti equispaziati.
26 # increasing=True genera le potenze in ordine crescente: 1, x, x2, ..., x8
27 V_eq = np.vander(x_eq, increasing=True)
28
29 # Esegue la fattorizzazione LU con pivot totale di una matrice quadrata A.
30 # Ritorna P, L, U, Q tali che P @ A @ Q = L @ U
31 # P = matrici di permutazione (riga)
32 # Q = matrici di permutazione (colonna)
33 def fatt_lu_pivot_totale(A):
34
35     n = A.shape[0]
36     # Inizializziamo le matrici di permutazione come matrici identità
37     P = np.eye(n)
38     Q = np.eye(n)
39
40     # Lavoriamo su una copia per non modificare la matrice originale
41     A_lu = A.copy().astype(float)
42
43     for k in range(n - 1):
44         # 1. RICERCA DEL PIVOT TOTALE
45         # Cerchiamo il massimo valore assoluto nella sottomatrice A_lu[k:n, k:n]
46         sub_matrix = np.abs(A_lu[k:n, k:n])
47         # r_rel e c_rel sono gli indici del massimo relativi alla sottomatrice
48         r_rel, c_rel = np.unravel_index(np.argmax(sub_matrix), sub_matrix.shape)
49
50         # Riportiamo gli indici assoluti nella matrice intera
51         r = r_rel + k
52         c = c_rel + k
53
54         # 2. SCAMBIO RIGHE (nella matrice A, e tracciamento in P)
55         if r != k:
56             A_lu[[k, r], :] = A_lu[[r, k], :]
57             P[[k, r], :] = P[[r, k], :]
58
59         # 3. SCAMBIO COLONNE (nella matrice A, e tracciamento in Q)
60         if c != k:
61             A_lu[:, [k, c]] = A_lu[:, [c, k]]
62             Q[:, [k, c]] = Q[:, [c, k]]
63
64         # Controllo tolleranza per evitare divisioni per zero
65         if np.abs(A_lu[k, k]) < 1e-15:
66             raise ValueError("La matrice ha un elemento pivotale quasi nullo. Sistema  
singolare o malcondizionato.")
```

```

67
68     # 4. CALCOLO DEI MOLTIPLICATORI E AGGIORNAMENTO matrice (Algoritmo di Gauss)
69     # Memorizziamo i moltiplicatori nella parte inferiore di A_lu
70     A_lu[k+1:n, k] = A_lu[k+1:n, k] / A_lu[k, k]
71     # Aggiorniamo la sottomatrice restante utilizzando il prodotto esterno
72     A_lu[k+1:n, k+1:n] -= np.outer(A_lu[k+1:n, k], A_lu[k, k+1:n])
73
74     # Estraiamo L e U dalla matrice combinata A_lu
75     # L ha i moltiplicatori sotto la diagonale e 1 sulla diagonale
76     L = np.tril(A_lu, -1) + np.eye(n)
77     # U ha gli elementi sulla diagonale e sopra
78     U = np.triu(A_lu)
79
80     return P, L, U, Q
81
82 # 1. Calcoliamo la fattorizzazione con il pivot totale sulla matrice di Vandermonde
83 P_eq, L_eq, U_eq, Q_eq = fatt_lu_pivot_totale(V_eq)
84
85 # 2. Risoluzione del sistema strutturato a passi:
86 # Passo a) Moltiplichiamo il termine noto per P: y_perm = P * y
87 # (Nota: fare P @ y_eq equivale matematicamente a fare P.T @ y_eq a seconda di come è
88 #   costruita P,
89 # avendo scambiato le righe di un'identità facciamo direttamente il prodotto della
90 #   matrice P ottenuta)
91 y_perm_eq = P_eq @ y_eq
92
93 # Passo b) Sostituzione in avanti per risolvere L * z = y_perm
94 z_eq = las.solve_triangular(L_eq, y_perm_eq, lower=True)
95
96 # Passo c) Sostituzione all'indietro per risolvere U * a_tilde = z
97 a_tilde = las.solve_triangular(U_eq, z_eq)
98
99 # Passo d) Poiché U * (Q.T * a) = P * y, allora a_tilde = Q.T * a.
100 # Per ricavare a dobbiamo moltiplicare per Q a sinistra: a = Q * a_tilde
101 a_eq = Q_eq @ a_tilde
102
103
104 # =====
105 # Punto 4: Nodi di Chebyshev e relativo polinomio
106 # =====
107
108 a, b = -2, 2
109 k = np.arange(n_punti)
110 # Formula per i Nodi di Chebyshev nell'intervallo [a, b]
111 x_cheb = (a + b + (b - a) * np.cos((2 * k + 1) * np.pi / (2 * n_punti))) / 2
112 y_cheb = f(x_cheb)
113
114 # Matrice di Vandermonde per i nodi di Chebyshev e risoluzione sistema (stesso iter di
115 #   prima)
116 V_cheb = np.vander(x_cheb, increasing=True)
117 P_cheb, L_cheb, U_cheb, Q_cheb = fatt_lu_pivot_totale(V_cheb)
118
119 y_perm_cheb = P_cheb @ y_cheb
120 z_cheb = las.solve_triangular(L_cheb, y_perm_cheb, lower=True)
121 a_tilde_cheb = las.solve_triangular(U_cheb, z_cheb)
122 a_cheb = Q_cheb @ a_tilde_cheb
123
124 # =====
125 # Punto 5: Valutazione dei polinomi e Grafico (550 punti)
126 # =====
127
128 # Generiamo i 550 punti equidistanti
129 x_plot = np.linspace(-2, 2, 550)
130 y_esatto = f(x_plot)
131
132 y_plot_eq = np.polyval(a_eq[:-1], x_plot)
133 y_plot_cheb = np.polyval(a_cheb[:-1], x_plot)
134
135 # Usiamo subplots per avere un controllo migliore sulla figura
136 fig, ax = plt.subplots(figsize=(10, 6))

```

```

137 # Spostiamo il grafico principale un po' verso destra per fare spazio ai bottoni
138 plt.subplots_adjust(left=0.25)
139
140 # 1. Disegniamo le linee e i nodi, SALVANDOLI in delle variabili.
141 l0, = ax.plot(x_plot, y_esatto, 'k-', label='f(x) Esatta', linewidth=2)
142 l1, = ax.plot(x_plot, y_plot_eq, 'r--', label='Equispaziati')
143 l2, = ax.plot(x_plot, y_plot_cheb, 'b-.', label='Chebyshev')
144
145 # Salviamo anche i nodi così li nascondiamo assieme alle curve
146 p1, = ax.plot(x_eq, y_eq, 'ro', markersize=6)
147 p2, = ax.plot(x_cheb, y_cheb, 'bs', markersize=6)
148
149 ax.set_title('Interpolazione: Equispaziati vs Chebyshev')
150 ax.set_xlabel('x')
151 ax.set_ylabel('y')
152 ax.set_ylim([-8, 8])
153 ax.grid(True)
154
155 # --- 2. Selezione bottoni ---
156 ax_check = plt.axes([0.02, 0.45, 0.18, 0.2])
157
158 # Rimuoviamo il riquadro nero attorno all'area dei bottoni
159 for spine in ax_check.spines.values():
160     spine.set_visible(False)
161 ax_check.set_facecolor('#f4f4f4') # Sfondo grigino elegante
162
163 labels = ['f(x) Esatta', 'Equispaziati', 'Chebyshev']
164 visibility = [True, True, True]
165 colori = ['black', 'red', 'blue']
166
167 # Moltiplichiamo per 3 le liste a singolo elemento per evitare l'errore del cyclar
168 check = CheckButtons(
169     ax=ax_check,
170     labels=labels,
171     actives=visibility,
172     # Stile del testo
173     label_props={'color': colori, 'fontweight': ['bold'] * 3, 'fontsize': [10] * 3},
174     # Stile dei quadratini
175     frame_props={'edgecolor': colori, 'facecolor': ['white'] * 3, 'linewidth': [1.5] * 3},
176     # Stile delle spunte "X"
177     check_props={'color': colori, 'linewidth': [2.5] * 3}
178 )
179
180 # --- 3. FUNZIONE DI AGGIORNAMENTO ---
181 def func(label):
182     if label == 'f(x) Esatta':
183         l0.set_visible(not l0.get_visible())
184     elif label == 'Equispaziati':
185         l1.set_visible(not l1.get_visible())
186         p1.set_visible(not p1.get_visible())
187     elif label == 'Chebyshev':
188         l2.set_visible(not l2.get_visible())
189         p2.set_visible(not p2.get_visible())
190
191     plt.draw()
192
193 check.on_clicked(func)
194
195
196
197 # =====
198 # Punto 6: Calcolo Condizionamento, Residuo ed Errore Massimo
199 # =====
200
201 # 1. Numero di condizionamento (cond) di Vandermonde, L e U
202 cond_V_eq, cond_V_cheb = np.linalg.cond(V_eq), np.linalg.cond(V_cheb)
203 cond_L_eq, cond_L_cheb = np.linalg.cond(L_eq), np.linalg.cond(L_cheb)
204 cond_U_eq, cond_U_cheb = np.linalg.cond(U_eq), np.linalg.cond(U_cheb)
205
206 # 2. Residuo del sistema lineare: norma di (V*a - y)
207 res_eq = np.linalg.norm(V_eq @ a_eq - y_eq)
208 res_cheb = np.linalg.norm(V_cheb @ a_cheb - y_cheb)

```

```

209
210 # 3. Errore massimo di interpolazione su 550 punti:  $\max|P(x) - f(x)|$ 
211 err_max_eq = np.max(np.abs(y_plot_eq - y_esatto))
212 err_max_cheb = np.max(np.abs(y_plot_cheb - y_esatto))
213
214
215
216 # 1. Definisco le righe di dati
217 righe = [
218     ["Equispaziati", cond_V_eq, cond_L_eq, cond_U_eq, res_eq, err_max_eq],
219     ["Chebyshev", cond_V_cheb, cond_L_cheb, cond_U_cheb, res_cheb, err_max_cheb]
220 ]
221
222 # 2. Definisco i nomi delle colonne
223 intestazioni = ["Metodo", "Cond(V)", "Cond(L)", "Cond(U)", "Residuo Sistema", "Errore Max
    (550 pt)"]
224
225 # 3. Stampa la tabella
226 print("\n--- TABELLA DI CONFRONTO ---")
227 # 'github' genera una tabella in stile Markdown (con la riga sotto l'intestazione)
228 # floatfmt=".5e" applica automaticamente la notazione scientifica a tutti i numeri.
229 print(tabulate(righe, headers=intestazioni, tablefmt="github", floatfmt=".5e"))
230 print("\n-- Chiudere la finestra del grafico per terminare il programma --\n")
231 plt.show()

```

Listing 1: Implementazione dell'interpolazione con Pivot Totale e confronto numerico.

3 Analisi dei Risultati Numerici

L'andamento grafico evidenzia chiaramente l'accuratezza dell'approssimazione mediante i nodi di Chebyshev, i quali riescono a contenere l'errore lungo tutto il dominio, a differenza dell'interpolante su nodi equispaziati che devia in prossimità di $x = \pm 2$ a causa del fenomeno di Runge.

Di seguito viene strutturata la tabella comparativa degli indici di stabilità e precisione ricavati dall'esecuzione dell'algoritmo.

3.1 Tabella Comparativa

Tabella 1: Confronto indici di condizionamento, residuo e errore analitico.

Distribuzione Nodi	$K(V)$	$K(L)$	$K(U)$	Residuo Sistema	Errore Max
Equispaziati	5.35×10^3	6.85×10^0	2.95×10^3	7.07×10^{-15}	2.09×10^{-2}
Chebyshev	2.41×10^3	1.20×10^1	2.21×10^3	6.83×10^{-15}	4.11×10^{-3}

3.2 Commento ai Dati

Dall'analisi della Tabella 1 emergono le seguenti considerazioni:

- **Residuo del Sistema Lineare:** In entrambi i casi il residuo $\|Va - y\|$ si attesta su valori prossimi alla precisione di macchina ($\approx 10^{-15}$). Questo dimostra che il sistema lineare viene risolto in modo formalmente corretto dal calcolatore, forzando il polinomio a passare esattamente per i nodi presi in esame.
- **Errore Massimo di Interpolazione:** Nonostante i residui siano equivalenti, l'errore massimo valutato sui 550 punti distorce la situazione. Per i nodi equispaziati l'errore assume proporzioni elevate a causa delle oscillazioni di Runge ai bordi, mentre per i nodi di Chebyshev l'errore rimane limitato, confermando la superiorità matematica di una scelta strategica dei punti di campionamento.
- **Condizionamento delle Matrici:** L'indice $K(V)$ della matrice equispaziata risulta significativamente superiore rispetto a quello ottenuto con i nodi di Chebyshev, riflettendo l'intrinseca vicinanza alla singolarità delle basi monomiali non opportunamente distribuite. L'introduzione del pivot totale garantisce che la scomposizione in fattori L e U preservi la massima stabilità algoritmica consentita dal problema.